Unit 3:

Classes and Objects in C++

- **Specifying a Class:**

A class is a user-defined data type that binds data and functions together. It serves as a blueprint to create objects.

Syntax:

```
class ClassName {
private:
    // data members
    // member functions

public:
    // data members
    // member functions
};
```

Example:

```
class Student {
private:
    int rollNo;
    float marks;

public:
    void input();
    void display();
};
```

Access Specifiers :

- Access specifiers define the visibility or accessibility of class members (variables and functions) from outside the class.

- C++ provides three main access specifiers:

1. Private
2. public
3. protected

1. private

- Members are accessible only within the class
- Default access level for class members

```
class Demo {
private:
    int a;  // only class functions can access this
};
```

2. public

Members are accessible from anywhere (including outside the class)

```
class Demo {
public:
    int x;  // accessible using object
};
```

3. protected

- Members are accessible within the class and its derived classes
- Not accessible from outside the class directly

```
class Base {
protected:
   int value;
};

class Derived : public Base {
public:
   void show() {
      cout << "Value: " << value;  //  ✅ allowed in derived class
   }
```

Access Specifier	Accessible Within Class	Accessible Outside Class	Accessible in Derived Class
`private`	✅ Yes	❌ No	❌ No
`protected`	✅ Yes	❌ No	✅ Yes
`public`	✅ Yes	✅ Yes	✅ Yes

```
#include <iostream>
using namespace std;
class Base {
private:
    int a = 10;      // only accessible inside Base
protected:
    int b = 20;      // accessible in Base and Derived
public:
    int c = 30;      // accessible from anywhere

    void showBase() {
        cout << "Private a: " << a << endl;
        cout << "Protected b: " << b << endl;
        cout << "Public c: " << c << endl;
    }
};
class Derived : public Base {
public:
    void showDerived() {
        // cout << "a: " << a; ❌ Not accessible (private)
        cout << "b: " << b << endl;  // ✅ Accessible (protected)
        cout << "c: " << c << endl;  // ✅ Accessible (public)
    }
};
int main() {
    Base obj;
    obj.showBase();        // ✅ allowed

    // cout << obj.a; ❌ Error (private)
    // cout << obj.b; ❌ Error (protected)
    cout << obj.c << endl;  // ✅ OK (public)

    Derived d;
    d.showDerived();       // ✅ Can access protected and
public

    return 0;
}
```

- **Defining Member Functions:**

Member functions can be defined inside or outside the class using the scope resolution operator ::.

Syntax (Outside class):

```
void ClassName::FunctionName() {
   // function body
}
```

```
#include <iostream>
using namespace std;

// Class Declaration
class Student {
private:
    int rollNo;
    float marks;

public:
    void input();    // Function Declaration
    void display();  // Function Declaration
};

// Member Function Definition (Outside Class)
void Student::input() {
    cout << "Enter Roll No and Marks: ";
    cin >> rollNo >> marks;
}
 void Student::display() {
cout << "Roll No: " << rollNo << ", Marks: " << marks <<
endl;
}

// Main Function
int main() {
    Student s1, s2; // Creating objects

    cout << "--- Enter details for Student 1 ---" << endl;
    s1.input();
    cout << "--- Enter details for Student 2 ---" << endl;
    s2.input();

    cout << "\n--- Student Details ---" << endl;
    s1.display();
    s2.display();

    return 0;
}
```

- **Nesting of Member Functions:**
  A member function can call another member function of the same class directly.

```
#include <iostream>
using namespace std;

// Class Definition
class Test {
private:
    int a, b;

    // Private Member Function
    int sum() {
        return a + b;
    }

public:
    // Public Member Function
    void input() {
        cout << "Enter two numbers: ";
        cin >> a >> b;
    }
    // Another Public Member Function calling a Private Member Function
    void display() {
        cout << "Sum is: " << sum() << endl;
    }
};

// Main Function
int main() {
    Test t;       // Creating object
    t.input();    // Calling input function
    t.display();  // Calls display which internally calls sum()

    return 0;
}
```

- Private member function `sum()` is **nested** inside public function `display()`

Private Member Functions:

- In C++, a private member function is a function declared inside the private: section of a class.
- It cannot be accessed directly using the object from main() or outside the class.
- Only other member functions of the same class can call it.
- This is useful when you want to hide internal details or helper logic from the outside world.

Example:

```
class Demo {
private:
    void showPrivate() {
        cout << "This is a private function" << endl;
    }

public:
    void callPrivate() {
        showPrivate();
    }
};
```

```
#include <iostream>
using namespace std;

class Demo {
private:
    // Private function - hidden from outside
    void showPrivate() {
        cout << "This is a private function." << endl;
    }

public:
    // Public function - accessible from main
    void callPrivate() {
        // This public function internally calls the private one
        showPrivate();  // ✅ This is allowed
    }
};

int main() {
    Demo d;

    // d.showPrivate(); ❌ Error: private function, cannot access from main

    d.callPrivate();    // ✅ This works, and it will call the private function
internally
```

- Output:

This is a private function.

- **Why Use** private function :
- Think of a private function like a **helper function**:
- You don't want the **outside world (main())** to call it directly.
- But your **class logic (public functions)** can use it.

Creating Objects & Memory Allocation:

- A class defines a blueprint.
- An object is an actual instance of a class.
- Memory is allocated only when an object is created, not when the class is defined.

Syntax:

ClassName obj1, obj2;

Example:

```
Student s1, s2;
s1.input();
s1.display();
```

Memory Allocation:
Memory is allocated only when the object is created, not when the class is defined.

```
#include <iostream>
using namespace std;

class Student {
public:
    int roll;
    void show() {
        cout << "Roll No: " << roll << endl;
    }
};

int main() {
    Student s1, s2; // Memory allocated here for roll in both s1 and s2

    s1.roll = 101;
    s2.roll = 102;

    s1.show();
    s2.show();

    return 0;
}
```

Static Data Members:

- A static data member is shared among all objects (common for all).
- A static member function can access only static data members.
- Static Members Shared by all objects; must define outside class.
- **Characteristics:**
- Belongs to the class, not to any specific object.
- Only one copy exists for all objects.
- Must be defined outside the class.
- Can be accessed using class name or object.

Example:

```
class Count {
private:
   static int cnt;                          // declaration inside class

public:
   void increase() {
      cnt++;
   }
   void display() {
      cout << "Count = " << cnt << endl;
   }
};

int Count::cnt = 0;                         // definition outside class
```

This means count is **common to all Counter objects**.

Wat is a Static Member Function:

A function that can access only static members of the class.

Characteristics:

- Can be called **without an object**, using the class name.
- It cannot access non-static members directly (since it does not work on a specific object).

Example:

```
static void showCount() {
cout << "Count = " << count << endl;   // ✅ accessing static data
}
```

```
class Counter {
private:
    static int count;  // Shared among all objects

public:
    void increment() {
        count++;       // Every object increases the same count
    }

    static void showCount() {
        cout << "Count = " << count << endl;
    }
};

// Define static member outside the class
int Counter::count = 0;

int main() {
    Counter a, b, c;

    a.increment();   // count = 1
    b.increment();   // count = 2
    c.increment();   // count = 3

    Counter::showCount();  // Output: Count = 3
```

Array of Objects:

- Objects can be stored in arrays like variables.
- You can create an array of objects just like arrays of integers. Each element is an object.

```
#include <iostream>
using namespace std;

class Item {
private:
    int code;

public:
    void getData() {
        cout << "Enter item code: ";
        cin >> code;
    }

    void showData() {
        cout << "Item code: " << code << endl;
    }
};
```

```
int main() {
    Item items[3];  // Array of 3 objects

    for (int i = 0; i < 3; i++) {
        items[i].getData();
    }

    for (int i = 0; i < 3; i++) {
        items[i].showData();
    }

    return 0;
}
```

Objects as Function Arguments:

Objects can be passed to functions by value or reference.

- You can pass an object to a function:

1. By Value: A copy is passed.
2. By Reference: The actual object is passed (more efficient, allows modification).

Example: Passing Object by Value

```
#include <iostream>
using namespace std;

class Student {
public:
    int roll;
    void setRoll(int r) {
        roll = r;
    }
    void show() {
        cout << "Roll: " << roll << endl;
    }
};

void printStudent(Student s) {  // Passed by value
    s.show();
}

int main() {
    Student s;
    s.setRoll(101);
    printStudent(s);
```

Example: Passing Object by Reference

```
#include <iostream>
using namespace std;

class Student {
public:
    int roll;
    void setRoll(int r) {
        roll = r;
    }
    void show() {
        cout << "Roll: " << roll << endl;
    }
};

void modify(Student &s) {          // Passed by reference
    s.roll = 202;                  // Modifies actual object
}

int main() {
    Student s;
    s.setRoll(101);
    modify(s);
    s.show(); // Output: Roll: 202
```

Constructors and Their Types:

Constructor:

A constructor is a special function automatically called when an object is created. It has the same name as the class.

Types of Constructors:

1. Default Constructor
2. Parameterized Constructor
3. Constructor Overloading
4. Constructors with Default Arguments
5. Dynamic Initialization of Objects

- A constructor is a special member function that:
- Has the same name as the class.
- Has no return type (not even void).
- Is called automatically when an object is created.

Why use a Constructor?

- To **initialize objects** with default or initial values.
- To **reduce repetitive code** when creating multiple objects with similar starting states.
- To ensure that an object is **in a valid state** immediately after creation.

1. Default Constructor:

- A constructor that takes no arguments.
- Automatically called when object is created.

Example:

```
#include <iostream>
using namespace std;

class Demo {
public:
    Demo() { // Default constructor
        cout << "Default Constructor called." << endl;
    }
};

int main() {
    Demo obj; // Constructor is called automatically
    return 0;
}
```

2. Parameterized Constructor:

- Accepts arguments to initialize class members.
- You can have multiple constructors with different parameter lists.

Example:

```
#include <iostream>
using namespace std;

class Rectangle {
private:
   int length, width;

public:
   Rectangle(int l, int w) { // Parameterized constructor
      length = l;
      width = w;
   }

   void area() {
      cout << "Area = " << length * width << endl;
   }
};

int main() {
   Rectangle r(10, 5); // Pass values at creation
   r.area();
```

3. Constructor Overloading:

- Multiple constructors with different parameter lists.
- Based on arguments passed, appropriate constructor is called.

Example:

```
#include <iostream>
using namespace std;
class Number {
private:
   int value;
public:
   Number() {              // Constructor 1: No argument
      value = 0;
   }
   Number(int v) {         // Constructor 2: One argument
      value = v;
   }
  void display() {
      cout << "Value = " << value << endl;
   }
};
int main() {
   Number n1;      // Calls default constructor
   Number n2(50);  // Calls parameterized constructor
   n1.display();
   n2.display();
```

4. Constructor with Default Arguments:

- Constructor that provides default values to parameters.
- User can call with fewer or no arguments.

Example:

```
#include <iostream>
using namespace std;

class Demo {
private:
   int x, y;

public:
   Demo(int a = 10, int b = 20) {
      x = a;
      y = b;
   }

   void display() {
      cout << "x = " << x << ", y = " << y << endl;
   }
};

int main() {
   Demo d1;         // Uses both default values
   Demo d2(100);    // Overrides first argument only
   Demo d3(5, 15);  // Overrides both arguments

   d1.display();
   d2.display();
   d3.display();

   return 0;
}
```

5. **Dynamic Initialization of Objects:**

- Initialize objects with values given at runtime.
- Constructor can accept input values or expressions.

Example:

```
#include <iostream>
using namespace std;

class Circle {
private:
   float radius, area;

public:
   Circle(float r) { // Dynamic initialization using constructor
      radius = r;
      area = 3.14 * radius * radius;
   }

   void display() {
      cout << "Radius = " << radius << ", Area = " << area << endl;
   }
};

int main() {
   float r;
   cout << "Enter radius: ";
   cin >> r;

   Circle c(r); // Value passed at runtime
   c.display();

   return 0;
```

Destructors:

- A destructor is a special member function that:
- Used to release memory/resources.
- Has the same name as the class, but starts with a tilde ~
- Has no arguments and no return type
- Is called automatically when an object goes out of scope or is explicitly deleted

Purpose of a Destructor:

- To release resources like:
- Memory allocated with new
- Open files or network connections
- To perform cleanup tasks before the object is destroyed.

Syntax:

```
~ClassName() {
    // cleanup code
}
```

Example:

```
class Sample {
public:
   Sample() {
      cout << "Constructor called" << endl;
   }
   ~Sample() {
      cout << "Destructor called" << endl;
```

Example: Destructor in Action

```
#include <iostream>
using namespace std;

class Demo {
public:
    Demo() {
        cout << "Constructor: Object is created." << endl;
    }

    ~Demo() {
        cout << "Destructor: Object is destroyed." << endl;
    }

    void show() {
        cout << "Inside show() function." << endl;
    }
};

int main() {
    cout << "--- Start of main() ---" << endl;

    {
        Demo d;  // Constructor is called here
        d.show();
    }  // Destructor is called automatically here when 'd' goes out of scope

    cout << "--- End of main() ---" << endl;
    return 0;
}
```

Output:

```
--- Start of main() ---
Constructor: Object is created.
Inside show() function.
Destructor: Object is destroyed.
--- End of main() ---
```

Example with Dynamic Memory (Why Destructors Are Important):

Dynamic Memory:

- When you create variables at runtime using the new keyword, memory is allocated from the heap.
- But you must manually free this memory using delete.
- If you don’t, your program will cause a memory leak (waste of memory).

```
#include <iostream>
using namespace std;

class Sample {
private:
   int* ptr;  // pointer to dynamically allocated memory

public:
   // Constructor: allocate memory
   Sample() {
      ptr = new int;      // dynamic memory allocation
      *ptr = 100;         // assign value
      cout << "Constructor: Memory allocated." << endl;
   }

   // Display value
   void display() {
      cout << "Value = " << *ptr << endl;
   }

   // Destructor: free memory
   ~Sample() {
      delete ptr;         // free memory
      cout << "Destructor: Memory released." << endl;
   }
};
int main() {
   Sample s1;      // Constructor called
   s1.display();   // Shows value
// When main ends, destructor is called automatically
   return 0;
```

Output:
Constructor: Memory allocated.
Value = 100
Destructor: Memory released.

if we didn’t write the destructor?
Then the memory allocated by new is never freed, even after the object is destroyed — this is a memory leak.

String Class :

- In C++, string is a class provided by the Standard Template Library (STL) that allows easy manipulation of sequences of characters.
- Unlike C-style strings (char arrays), C++ string objects:
- Grow/shrink dynamically
- Have built-in functions for easy manipulation
- Avoid buffer overflows

How to Use the String Class:

To use string objects, include the <string> header and use the std namespace:

```
#include <string>
using namespace std;
```

Syntax: Declaring String Objects:

```
string str1;                // Empty string
string str2 = "Hello";      // Initialized string
string str3("World");       // Another way to initialize
```

```
using namespace std;

int main() {
    // Declaring string objects
    string name, city, greeting;

    // Input from user
    cout << "Enter your name: ";
    getline(cin, name);  // Accepts full name including spaces

    cout << "Enter your city: ";
    getline(cin, city);

    // Assign and manipulate strings
    greeting = "Hello, " + name + " from " + city + "!";

    // Display the greeting
    cout << greeting << endl;

    // Accessing individual characters
    cout << "First letter of your name: " << name[0] << endl;

    // Length of string
    cout << "Your name has " << name.length() << " characters." << endl;

    return 0;
```

`getline(cin, name);`	Reads a full line including spaces
`greeting = "Hello, " + name + ...`	Strings are concatenated using `+`
`name[0]`	Accesses the first character in the string
`name.length()`	Returns the number of characters in the string

String Manipulation:

String manipulation means performing operations on string objects like:

1. Appending (adding more text)
2. Inserting, replacing
3. Finding substrings
4. Erasing characters
5. Getting substrings
6. Changing case, etc.

- C++ provides many built-in functions for string manipulation in the <string> header.

1. Concatenation:

```
string result = str1 + " " + str2;
cout << "Concatenated: " << result << endl;
```

- Joins str1, a space " ", and str2
- result = "Hello World"

Output: Concatenated: Hello World

2. **Append:**

```
result.append("!");
cout << "After append: " << result << endl;
```

- Adds "!" to the end of result
- result = "Hello World!"

Output: After append: Hello World!

3. Insert:

```
result.insert(5, ", C++");
cout << "After insert: " << result << endl;
```

Inserts ", C++" at index 5 (after "Hello")
"Hello World!" → "Hello, C++ World!"

Output: After insert: Hello, C++ World!

4. Replace:

```
result.replace(6, 4, "Language");
cout << "After replace: " << result << endl;
```

Replaces 4 characters starting from index 6 with "Language"

- "Hello, C++ World!" →
- From index 6, 4 characters = ", C+"
- Replace it with "Language"
- Result = "Hello Language World!"

Output:After replace: Hello Language World!

5. Erase:

```
result.erase(13, 8);
cout << "After erase: " << result << endl;
```

- From index 13, erase 8 characters
- "Hello Language World!" → remove " World!"
- Result = "Hello Language!"

Output: After erase: Hello Language!

6. Substring:

```
string part = result.substr(0, 5);
cout << "Substring (0-5): " << part << endl;
```

- Extracts substring from index 0, length 5
- From "Hello Language!" → "Hello"

Output: Substring (0-5): Hello

7. Find:

```
int pos = result.find("World");
```

- Tries to find "World" in "Hello Language!"
- Not found → find() returns string::npos

Output: "World" not found.

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    string str1 = "Hello";
    string str2 = "World";

    // 1. Concatenation
    string result = str1 + " " + str2;
    cout << "Concatenated: " << result << endl;

    // 2. Append another string
    result.append("!");
    cout << "After append: " << result << endl;

    // 3. Insert characters
    result.insert(5, ", C++");
    cout << "After insert: " << result << endl;

    // 4. Replace a part of the string
    result.replace(6, 4, "Language");
    cout << "After replace: " << result << endl;

    // 5. Erase characters
    result.erase(13, 8);
    cout << "After erase: " << result << endl;

    // 6. Substring
    string part = result.substr(0, 5);
    cout << "Substring (0-5): " << part << endl;

    // 7. Find
    int pos = result.find("World");
    if (pos != string::npos)          // check if found
        cout << "\"World\" found at position: " << pos << endl;
    else
        cout << "\"World\" not found." << endl;

    return 0;
}
```

//Output:

```
Concatenated: Hello World
After append: Hello World!
After insert: Hello, C++ World!
After replace: Hello Language World!
After erase: Hello Language!
Substring (0-5): Hello
"World" not found.
```

Function	Purpose	Example
`+` or `.append()`	Adds strings	`"abc" + "def"` → `"abcdef"`
`.insert(pos, str)`	Inserts string at given position	`s.insert(3, "hi")`
`.replace(pos, len, str)`	Replaces part of string	`s.replace(1,2,"hi")`
`.erase(pos, len)`	Removes part of string	`s.erase(2, 3)`
`.substr(pos, len)`	Extracts substring	`s.substr(0, 5)`
`.find(str)`	Searches for substring	Returns position or `string::npos`

- **Relational Operations:**
- Relational operations compare two strings using operators like:

1. == → Equal to
2. != → Not equal to
3. < → Less than
4. > → Greater than
5. <= → Less than or equal to
6. >= → Greater than or equal to

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1, str2;

    cout << "Enter first string: ";
    getline(cin, str1);

    cout << "Enter second string: ";
    getline(cin, str2);

    // 1. Equality Check
    if (str1 == str2)
        cout << "Both strings are equal." << endl;
    else
        cout << "Strings are not equal." << endl;

    // 2. Not equal
    if (str1 != str2)
        cout << "str1 is not equal to str2." << endl;

    // 3. Less than
    if (str1 < str2)
        cout << "str1 is less than str2." << endl;

    // 4. Greater than
    if (str1 > str2)
        cout << "str1 is greater than str2." << endl;

    // 5. Less than or equal
    if (str1 <= str2)
        cout << "str1 is less than or equal to str2." << endl;

    // 6. Greater than or equal
    if (str1 >= str2)
        cout << "str1 is greater than or equal to str2." << endl;

    return 0;
}
```

```
Enter first string: Meenu
Enter second string: Joshi
Strings are not equal.
str1 is not equal to str2.
str1 is greater than str2.
str1 is greater than or equal to str2.
```

String characteristics (size,length,capacity):

1. size() and length()

- Both return the number of characters in the string.
- They are functionally the same.

2. capacity()

- Returns the total allocated space for the string (in characters).
- It is usually more than or equal to the actual size, so that the string can grow without reallocating memory each time.

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str = "Hello";

    cout << "String: " << str << endl;

    // Size and Length
    cout << "Size: " << str.size() << endl;
    cout << "Length: " << str.length() << endl;

    // Capacity
    cout << "Capacity: " << str.capacity() << endl;

    // Append more characters to see how capacity changes
    str += " World! This is a test to check capacity.";
    cout << "\nAfter appending text:\n";
    cout << "String: " << str << endl;
    cout << "Size: " << str.size() << endl;
    cout << "Capacity: " << str.capacity() << endl;

    // Resize the string manually
    str.resize(10);  // Trims string to 10 characters
    cout << "\nAfter resize(10):\n";
    cout << "String: " << str << endl;
    cout << "Size: " << str.size() << endl;
    cout << "Capacity: " << str.capacity() << endl;

    return 0;
}
```

Accessing characters in strings :

1. at()
2. substr()
3. find()
4. find_first_of()
5. find_last_of()

These functions are powerful tools for handling string data in C++.

1. at(index)

Returns the character at a specific position with bounds checking.

2. substr(start, length)

Extracts a substring starting from start with length characters.

3. find("text")

Returns the position of the first occurrence of "text" in the string.

4. find_first_of("chars")

Returns the first position where any character from "chars" is found.

5. find_last_of("chars")

Returns the last position where any character from "chars" is found.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string message = "Hello, Programming World!";

    // 1. at()
    cout << "Character at index 0 using at(): " << message.at(0)
<< endl;

    // 2. substr()
    string sub = message.substr(7, 11);  // From index 7, take 11
characters
    cout << "Substring (7, 11): " << sub << endl;

    // 3. find()
    size_t pos = message.find("World");
    if (pos != string::npos)
        cout << "\"World\" found at position: " << pos << endl;
    else
        cout << "\"World\" not found." << endl;

// 4. find_first_of()
    size_t first_vowel = message.find_first_of("AEIOUaeiou");
    if (first_vowel != string::npos)
        cout << "First vowel found at position: " << first_vowel
            << " ('" << message[first_vowel] << "')" << endl;

    // 5. find_last_of()
    size_t last_vowel = message.find_last_of("AEIOUaeiou");
    if (last_vowel != string::npos)
        cout << "Last vowel found at position: " << last_vowel
            << " ('" << message[last_vowel] << "')" << endl;

    return 0;
}
```

Output

```
Character at index 0 using at(): H
Substring (7, 11): Programming
"World" found at position: 19
First vowel found at position: 1 ('e')
Last vowel found at position: 20 ('o')
```

Comparing and swapping strings :

- compare()
- swap()

1. compare() Function

The compare() function is used to compare two strings lexicographically (dictionary order).

Syntax:

```
str1.compare(str2)
```

Return Values:

Value	Meaning
0	Both strings are equal
<0	str1 is less than str2
>0	str1 is greater than str2

2. swap() Function

The swap() function exchanges the contents of two strings.

Syntax:

```
str1.swap(str2);
```

```
string s1 = "Apple";
string s2 = "Banana";

// Comparing Strings
int result = s1.compare(s2);

cout << "Comparing \"" << s1 << "\" and \"" << s2 << "\":" << endl;
if (result == 0)
   cout << "Both strings are equal." << endl;
else if (result < 0)
   cout << "\"" << s1 << "\" is less than \"" << s2 << "\"." << endl;
else
   cout << "\"" << s1 << "\" is greater than \"" << s2 << "\"." << endl;

// Swapping Strings
cout << "\nBefore swap:" << endl;
cout << "s1 = " << s1 << endl;
cout << "s2 = " << s2 << endl;

s1.swap(s2);  // Swapping contents

cout << "\nAfter swap:" << endl;
cout << "s1 = " << s1 << endl;
cout << "s2 = " << s2 << endl;
```

Output:

Comparing "Apple" and "Banana":
"Apple" is less than "Banana".

Before swap:
s1 = Apple
s2 = Banana

After swap:
s1 = Banana
s2 = Apple

Without Using String functions

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1, str2;
    char ch;

    cout << "Enter first string: ";
    getline(cin, str1);

    cout << "Enter second string: ";
    getline(cin, str2);

    // a) Concatenation (manual)
    string concat = str1;
    for (int i = 0; i < str2.size(); i++) {
        concat += str2[i];
    }
    cout << "Concatenation: " << concat << endl;

    // b) Comparison (manual)
    int cmp = 0;
    int n = (str1.size() < str2.size()) ? str1.size() : str2.size();
    for (int i = 0; i < n; i++) {
    // c) Find character
    cout << "Enter a character to search in str1: ";
    cin >> ch;
    int pos = -1;
    for (int i = 0; i < str1.size(); i++) {
        if (str1[i] == ch) {
            pos = i;
            break;
        }
    }
    if (pos != -1)
        cout << "Character '" << ch << "' found at position: " << pos <<
endl;
    else
        cout << "Character '" << ch << "' not found." << endl;

    // d) Reverse (manual)
    string rev = "";
    for (int i = str1.size() - 1; i >= 0; i--) {
        rev += str1[i];
    }
    cout << "Reverse of str1: " << rev << endl;

    return 0;
}
```